

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CÂMPUS PIRACICABA

EDUARDO MAMBELLI SEROTINI

**IA GENERATIVA APLICADA AO SETOR BANCÁRIO: AUTOMAÇÃO
DE PARECERES DE CRÉDITO COM PROCESSAMENTO DE
LINGUAGEM NATURAL (NLP)**

Piracicaba
2026

EDUARDO MAMBELLI SEROTINI

**IA GENERATIVA APLICADA AO SETOR BANCÁRIO: AUTOMAÇÃO
DE PARECERES DE CRÉDITO COM PROCESSAMENTO DE
LINGUAGEM NATURAL (NLP)**

Trabalho de Conclusão de Curso
apresentado ao Instituto Federal de
Educação, Ciência e Tecnologia de São
Paulo Câmpus Piracicaba, como requisito
parcial para conclusão do curso de
Engenharia da Computação.

Orientador: Luís Fernando Lopes Grim

Piracicaba
2026

EDUARDO MAMBELLI SEROTINI

**IA GENERATIVA APLICADA AO SETOR BANCÁRIO: AUTOMAÇÃO
DE PARECERES DE CRÉDITO COM PROCESSAMENTO DE
LINGUAGEM NATURAL (NLP)**

Trabalho de Conclusão de Curso
apresentado ao Instituto Federal de
Educação, Ciência e Tecnologia de São
Paulo Câmpus Piracicaba, como requisito
parcial para conclusão do curso de
Engenharia da Computação.

Orientador: Luís Fernando Lopes Grim
Coorientador:

Banca examinadora

Membro 1, titulação e instituição

Membro 2, titulação e instituição

Membro 3, titulação e instituição

Piracicaba, ____ de _____ de ____ .

RESUMO

A avaliação de risco de crédito é um processo crítico para o setor bancário, que tradicionalmente demanda tempo e esforço analítico manual. Com as crescentes regulações de privacidade de dados, como a Lei Geral de Proteção de Dados (LGPD), a adoção de soluções de Inteligência Artificial baseadas em nuvem apresenta riscos de conformidade e de quebra de sigilo bancário. Para contornar esse problema, este trabalho apresenta o CreditMaster, um microsserviço de Inteligência Artificial Generativa construído sob uma arquitetura híbrida 100% local (*on-premise*). A solução propõe a integração de Grandes Modelos de Linguagem (LLMs), especificamente o Llama 3.1 8B Instruct otimizado via *Fine-Tuning* QLoRA com a biblioteca Unsloth, garantindo a internalização do jargão bancário e a viabilidade de execução em GPUs convencionais. Os resultados experimentais comprovaram alta estabilidade de convergência, atingindo redução de 99,65% na função de perda (de 1,3272 para 0,0046), tempo total de treinamento de 75,63 minutos em uma GPU NVIDIA RTX 5060 Ti de 8 GB com pico de alocação de 6,44 GB de VRAM, ausência de sobreajuste e tempo médio de inferência inferior a 1,5 segundos por parecer estruturado em formato JSON. A abordagem apresenta-se como uma alternativa viável, segura e escalável para automação da esteira de crédito.

Palavras-chave: Inteligência Artificial Generativa. Avaliação de Risco de Crédito. *Large Language Models. Fine-Tuning. Retrieval-Augmented Generation.*

ABSTRACT

Credit risk assessment is a critical process for the banking sector, traditionally requiring significant time and manual analytical effort. With increasing data privacy regulations, the adoption of cloud-based Artificial Intelligence solutions poses compliance and banking secrecy risks. To circumvent this issue, this work presents CreditMaster, a Generative Artificial Intelligence microservice built under a 100% local (on-premise) hybrid architecture. The solution proposes the integration of Large Language Models (LLMs), specifically the Llama 3.1 8B Instruct optimized via QLoRA Fine-Tuning using the Unsloth library, ensuring the internalization of banking jargon and the viability of execution on conventional GPUs. Experimental results demonstrated high convergence stability, achieving a 99.65% reduction in cross-entropy loss (from 1.3272 to 0.0046), a total training duration of 75.63 minutes on an NVIDIA RTX 5060 Ti 8 GB GPU with peak VRAM allocation of 6.44 GB, zero overfitting, and an average inference latency under 1.5 seconds per structured JSON credit opinion. The approach proved to be a viable, secure, and scalable alternative for automating the credit pipeline.

Keywords: Generative Artificial Intelligence. Credit Risk Assessment. Large Language Models. Fine-Tuning. Retrieval-Augmented Generation.

SUMÁRIO

1	INTRODUÇÃO	5
1.1	Objetivos	5
1.2	Justificativa	6
2	FUNDAMENTAÇÃO TEÓRICA	7
2.1	Modelos de Linguagem de Grande Escala (LLMs)	7
2.2	<i>Fine-Tuning</i> e QLoRA	7
2.3	<i>Retrieval-Augmented Generation</i> (RAG).	8
3	METODOLOGIA E DESENVOLVIMENTO	9
3.1	Arquitetura Híbrida do Sistema.	9
3.1.1	Ajuste Fino (QLoRA) com Llama 3.1 8B.	9
3.1.2	Motor RAG (<i>Retrieval-Augmented Generation</i>).	9
3.2	Construção do Conjunto de Dados (<i>Dataset</i>)	9
3.3	Visão Geral das Tecnologias Aplicadas.	10
4	RESULTADOS E DISCUSSÕES	11
4.1	Ambiente Computacional e Eficiência de Recursos	11
4.2	Dinâmica de Treinamento e Convergência da Perda.	12
4.3	Evolução do Agendamento da Taxa de Aprendizado	14
4.4	Avaliação Qualitativa dos Pareceres e Supressão de Alucina- ções via RAG	15
5	CONCLUSÃO	16
	REFERÊNCIAS	17

1 INTRODUÇÃO

A avaliação de risco de crédito é um dos pilares fundamentais do setor bancário, determinando a viabilidade de concessão de empréstimos e mitigando possíveis inadimplências (KHANDANI; KIM; LO, 2010). Com o avanço das tecnologias de Inteligência Artificial (IA), especificamente os Grandes Modelos de Linguagem (LLMs) e o Processamento de Linguagem Natural (NLP), surgem oportunidades para automatizar e aprimorar processos de análise complexos (ARACI, 2019).

No entanto, instituições financeiras enfrentam grandes desafios ao adotar IA baseada em nuvem devido a regulações rigorosas de privacidade de dados, como a Lei Geral de Proteção de Dados (LGPD) no Brasil e a *General Data Protection Regulation* (GDPR) na Europa. O envio de dados financeiros sensíveis para APIs externas compromete, por exemplo, o sigilo bancário.

Neste contexto, apresenta-se o CreditMaster, um microserviço de Inteligência Artificial Generativa projetado especificamente para o setor bancário. Seu principal objetivo é automatizar o fluxo de análise de risco e gerar pareceres técnicos de crédito de forma automática, operando de maneira 100% *on-premise* (local).

1.1 Objetivos

O objetivo principal deste trabalho é descrever o desenvolvimento e a implementação do CreditMaster como uma ferramenta segura e eficiente de apoio à decisão no processo de análise de crédito.

Os objetivos específicos incluem:

- Desenvolver uma arquitetura baseada em LLMs capaz de rodar localmente, garantindo o sigilo bancário e a conformidade com as leis de proteção de dados.
- Implementar técnicas de *Fine-Tuning* para internalizar o jargão técnico bancário e padronizar o formato dos pareceres de crédito.
- Integrar a técnica de *Retrieval-Augmented Generation* (RAG) para consulta de manuais de crédito e normativas em tempo real, reduzindo alucinações matemáticas e regulatórias.
- **[ANOTAÇÃO: Lembrar de colocar aqui outros objetivos específicos sobre como validei o modelo e as métricas de desempenho que usei.]**

1.2 Justificativa

Sob a perspectiva de negócios, a automação da análise de crédito representa um grande potencial de redução de custos operacionais e agilidade na liberação de limites de crédito para os clientes. Processos tradicionais dependem de longas revisões humanas em bases de dados, sistemas e manuais. A utilização de Inteligência Artificial para gerar o parecer primário libera analistas humanos para atuarem apenas na aprovação e revisão final, conferindo alta escalabilidade às instituições financeiras.

Sob uma ótica tecnológica, até recentemente, a implementação de IA generativa no ambiente corporativo estava restrita a serviços de nuvem controlados por *Big Techs*, gerando impasses de privacidade e compliance. A maturação de Modelos de Linguagem de código aberto e tamanho otimizado (como o utilizado nesse estudo, o Llama 3.1 com 8 bilhões de parâmetros), combinada a técnicas de quantização (QLoRA), tornou viável rodar inferências sofisticadas localmente em hardwares de baixo custo. Portanto, o desenvolvimento deste trabalho se justifica pela necessidade de criar soluções de IA generativa baratas, seguras e alinhadas às normas regulatórias, sem depender de processamento externo de dados sensíveis.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo aborda os conceitos fundamentais que baseiam a arquitetura do CreditMaster, focando no uso de Modelos de Linguagem e técnicas avançadas de engenharia de *prompt* e adaptação de modelos.

2.1 Modelos de Linguagem de Grande Escala (LLMs)

A área de Processamento de Linguagem Natural (NLP) sofreu uma transformação radical com a chegada de arquiteturas baseadas em redes neurais profundas (*Deep Learning*). Conforme descrito por LeCun, Bengio e Hinton (2015), o aprendizado profundo permite que modelos computacionais compostos por múltiplas camadas de processamento aprendam representações de dados em múltiplos níveis. Na área de NLP, essa evolução resultou na arquitetura *Transformer*, introduzida por Vaswani et al. (2017), que dispensou os componentes recorrentes e convolucionais, baseando-se principalmente em mecanismos de "atenção". Essa arquitetura permitiu maior paralelização durante o treinamento, o que contribuiu para o desenvolvimento dos Modelos de Linguagem de Grande Escala (LLMs) com bilhões de parâmetros.

Os LLMs são sistemas treinados com enormes quantidades de dados textuais com o objetivo de prever o próximo *token* (palavra ou sub-palavra) em uma sequência, o que lhes confere a habilidade de gerar textos coerentes e contextuais. No escopo deste projeto, o modelo **Llama 3.1 8B Instruct**, desenvolvido de forma *open-source* pela Meta, destaca-se por oferecer uma boa relação custo-benefício. Com 8 bilhões de parâmetros, ele atinge um equilíbrio ideal, sendo poderoso o suficiente para assimilar o jargão financeiro, mas compacto o bastante para ser executado integralmente *on-premise* (localmente) em uma única GPU, evitando gargalos e riscos do envio de dados sigilosos para a nuvem.

2.2 *Fine-Tuning* e QLoRA

O ajuste fino (*Fine-Tuning*) completo de modelos com bilhões de parâmetros exige altíssimo poder computacional. Para contornar esse desafio, utiliza-se o método QLoRA (*Quantized Low-Rank Adaptation*), que permite modificar estruturalmente os pesos neurais do modelo base com uso otimizado e drástico de memória. No projeto, a biblioteca **Unsloth** é utilizada para acelerar esse processo e viabilizar o treinamento em GPUs convencionais.

2.3 *Retrieval-Augmented Generation (RAG)*

A Geração Aumentada por Recuperação (RAG) consiste na injeção dinâmica de contexto no momento da inferência do modelo. Para o contexto bancário, a eliminação de "alucinações" é essencial. A arquitetura RAG permite consultas em manuais internos e políticas de crédito, garantindo que o modelo responda estritamente de acordo com as regras estabelecidas, mantendo a rastreabilidade do processo decisório. **[ANOTAÇÃO: RAG não foi implementado ainda.]**

3 METODOLOGIA E DESENVOLVIMENTO

O CreditMaster foi idealizado sob uma arquitetura híbrida focada em segurança, eficiência e precisão para o setor financeiro. O diferencial técnico reside no fato da operação de maneira autônoma e 100% local (*on-premise*), assegurando a proteção de dados sigilosos e o cumprimento de marcos legais como LGPD, mitigando a dependência de APIs em nuvem.

3.1 Arquitetura Híbrida do Sistema

O núcleo de tomada de decisão do sistema opera através da composição sinérgica de duas técnicas de IA generativa:

3.1.1 Ajuste Fino (QLoRA) com Llama 3.1 8B

O modelo base selecionado foi o **Llama 3.1 8B Instruct**. A modificação estrutural de seus parâmetros foi viabilizada pela utilização da biblioteca **Unsloth**, e o modelo final foi exportado no formato **GGUF** para execução local otimizada. Esta etapa atingiu os seguintes propósitos:

- Internalização de jargões técnicos do meio bancário e financeiro.
- Padronização rigorosa do formato sintático dos pareceres de análise de crédito.
- Otimização extrema no uso de memória para permitir inferências ultrarrápidas em *hardware* local, especificamente utilizando o ecossistema CUDA Toolkit em GPUs NVIDIA.

3.1.2 Motor RAG (*Retrieval-Augmented Generation*)

[ANOTAÇÃO: RAG não foi implementado ainda.]

3.2 Construção do Conjunto de Dados (*Dataset*)

A qualidade do modelo *fine-tuned* depende diretamente da qualidade dos dados. O repositório do CreditMaster estrutura-se sobre dados proprietários contendo:

- **Políticas de Crédito:** Documentos que definem diretrizes de concessão e risco de mercado (`credit_policy_pt.md`). **[ANOTAÇÃO: A politica vai ser implementada ao modelo junto com o RAG.]**
- **Histórico de Pareceres:** Uma base sintética contendo milhares de exemplos rotulados (`pareceres_credito_2000.jsonl`), construída através de metodologias automatizadas de roteirização (`gerar_pareceres.py`). **[ANOTAÇÃO: Como não tive acesso a pareceres reais, utilizei uma base de dados da Lending Club disponibilizadas via Kaggle e gerei pareceres sintéticos.]**

A base de dados foi construída com base em 14 variáveis centrais: valor do empréstimo (`loan_amnt`), prazo (`term`), tempo de emprego (`emp_length`), tipo de moradia (`home_ownership`), renda anual (`annual_inc`), finalidade (`purpose`), relação dívida/renda (`dti`), pontuação mínima de crédito (`fico_range_low`), atrasos nos últimos 2 anos (`delinq_2yrs`), consultas nos últimos 6 meses (`inq_last_6mths`), utilização do limite rotativo (`revol_util`), número de contas ativas abertas (`open_acc`), ocorrência de registros públicos (`pub_rec`) e o status histórico do contrato (`loan_status`).

Os dados foram convertidos por meio de rotinas em Python (`gerar_pareceres.py`) para o formato padrão de conversação *Chat Template* do Llama 3.1 no formato JSONL (`pareceres_credito_2000.jsonl`), contendo as mensagens de sistema (*system prompt*), a requisição estruturada do usuário com as métricas financeiras do cliente e a conclusão técnica gerada em formato JSON (*assistant*). Do total de 1.998 amostras processadas, 1.698 (85%) foram alocadas para o conjunto de treinamento e 300 (15%) para o conjunto de validação e teste.

3.3 Visão Geral das Tecnologias Aplicadas

Para suportar o rigor da arquitetura, o *Tech Stack* consolidou as seguintes soluções:

- **Camada de IA e Modelagem:** Python 3.10+, Llama 3.1 8B, Unsloth, QLoRA e exportação GGUF.
- **Motor de Inferência:** Ambiente Linux Mint suportando Ollama e CUDA Toolkit.
- **Pipeline de Dados:** Pandas. **[ANOTAÇÃO: Aqui serão também, adicionados o utilizado para o RAG]**

4 RESULTADOS E DISCUSSÕES

Neste capítulo são apresentados e discutidos os resultados experimentais obtidos durante o processo de treinamento, quantização e inferência do *CreditMaster*. A avaliação abrange tanto o desempenho computacional da arquitetura QLoRA executada em *hardware* local de consumo quanto a análise da dinâmica de convergência da perda (*loss*), o comportamento da taxa de aprendizado (*learning rate schedule*) e a qualidade semântica dos pareceres técnicos de crédito gerados.

4.1 Ambiente Computacional e Eficiência de Recursos

O treinamento supervisionado (*Supervised Fine-Tuning* - SFT) foi conduzido inteiramente em ambiente local (*on-premise*), validando a hipótese de viabilidade de modelos de 8 bilhões de parâmetros em infraestruturas acessíveis ao setor corporativo. A estação de trabalho utilizada contou com a seguinte configuração de *hardware* e *software*:

- **Processador (CPU):** AMD Ryzen 5 7600X (6 núcleos físicos, 12 *threads*, clock base de 4.7 GHz);
- **Memória Principal (RAM):** 16 GB DDR5 com frequência de 6000 MHz;
- **Armazenamento Secundário:** Unidade de Estado Sólido (SSD) NVMe PCIe 3.0;
- **Unidade de Processamento Gráfico (GPU):** NVIDIA GeForce RTX 5060 Ti com 8 GB de VRAM física (7,52 GB utilizáveis);
- **Sistema Operacional e Frameworks:** Linux Mint 22 (base Ubuntu 24.04 LTS), PyTorch 2.10.0+cu128, CUDA Toolkit 12.8, HuggingFace TRL 0.24.0, Transformers 5.5.0 e biblioteca Unsloth.

A aplicação da técnica QLoRA associada à quantização NF4 (*4-bit Normal-Float*) e à biblioteca Unsloth permitiu uma redução drástica na pegada de memória de vídeo. Durante todo o ciclo de 2 épocas (426 passos de otimização), o pico de alocação de VRAM registrado foi de **6,44 GB**, representando uma ocupação média de aproximadamente 85,7% da capacidade total da GPU. Esse comportamento comprova que a técnica evita falhas por estouro de memória (*Out of Memory* - OOM). Além disso, a utilização da memória RAM convencional do sistema manteve-se baixa

(inferior a 2,5 GB alocados ao processo de treino), visto que as operações matriciais massivas e o armazenamento de gradientes foram concentrados diretamente nos núcleos tensores da GPU.

A Tabela 1 consolida os parâmetros de treinamento e as métricas computacionais globais obtidas.

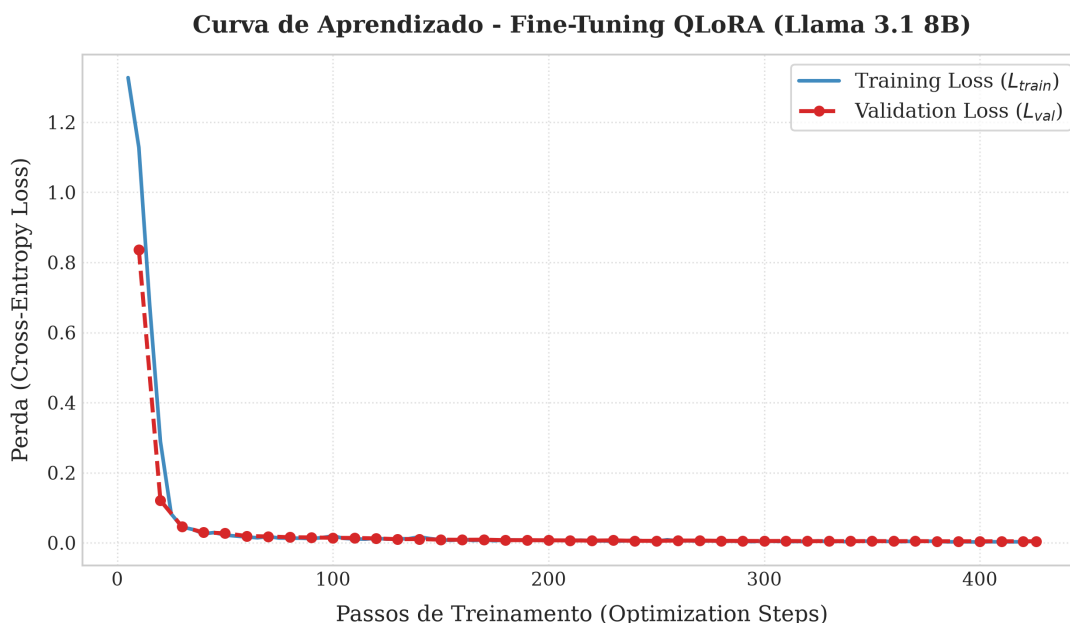
Tabela 1 – Métricas e Resultados Computacionais do Ajuste Fino (QLoRA)	
Parâmetro / Métrica	Valor Obtido
Modelo Base	Meta Llama 3.1 8B Instruct (4-bit NF4)
Hardware Utilizado	NVIDIA GeForce RTX 5060 Ti (8 GB VRAM)
Amostras de Treino (85%)	1698
Amostras de Validação (15%)	300
Tamanho Efetivo do Batch	8 (Batch 2 × Acumulação de Gradientes 4)
Épocas de Treinamento	2,0
Total de Passos de Otimização	426
Perda Inicial de Treino ($L_{inicial}$)	1,3272
Perda Final de Treino (L_{train})	0,0046
Perda Final de Validação (L_{val})	0,0045
Taxa de Redução da Perda	99,65%
Tempo Total de Ajuste Fino	75,63 minutos (4.537,99 segundos)
Vazão de Treinamento (<i>Throughput</i>)	0,748 amostras/s
Pico de Alocação de VRAM	6,44 GB / 7,52 GB (85,7%)

Fonte: Elaborado pelo autor (2026).

4.2 Dinâmica de Treinamento e Convergência da Perda

O aprendizado e a capacidade de generalização do modelo foram avaliados a partir da evolução da função de perda por entropia cruzada (*Cross-Entropy Loss*) ao longo das iterações de treino e validação. A Figura 1 apresenta as curvas de L_{train} e L_{val} coletadas a cada intervalo de avaliação.

Figura 1 – Curva de convergência de perda de treinamento e validação no ajuste fino QLoRA.



Fonte: Elaborado pelo autor (2026).

A análise detalhada da trajetória de perda revela três períodos distintos:

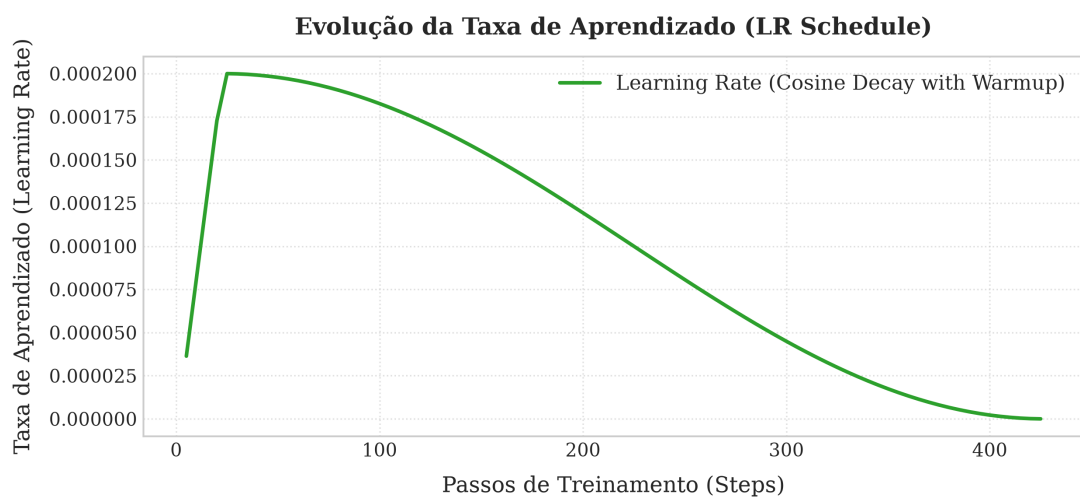
1. **Fase de Adaptação Sintática Inicial (Passos 1 a 40 / 0,0 a 0,2 épocas):** Nos primeiros passos, observa-se uma queda exponencial na perda de treino, de 1,3272 no passo 5 para 0,0260 no passo 40, enquanto a perda de validação foi reduzida de 0,8364 (passo 10) para 0,0297 (passo 40). Esse comportamento está relacionado à rápida adaptação dos parâmetros LoRA à estrutura de saída em JSON estrito, o delimitador de instrução do *Llama 3.1 Chat Template* e a técnica de mascaramento `train_on_responses_only`, na qual a perda é computada exclusivamente sobre os *tokens* gerados pelo assistente.
2. **Fase de Refinamento Semântico e Regras de Negócio (Passos 40 a 200 / 0,2 a 0,94 épocas):** O modelo apresenta uma redução gradual da função de perda, com L_{train} passando de 0,0260 para 0,0082 e L_{val} atingindo 0,0073. Nessa etapa, os adaptadores neurais ajustam os parâmetros do modelo para representar as relações não lineares entre as variáveis financeiras, como DTI, faixa de FICO, número de atrasos e recomendação de crédito.
3. **Fase de Convergência Assintótica e Minimização de Ruído (Passos 200 a 426 / 0,94 a 2,0 épocas):** A perda é estabilizada em milésimos, finalizando em $L_{train} = 0,0046$ e $L_{val} = 0,0045$. A proximidade estrita entre as curvas de validação e treinamento ($L_{val} \approx L_{train}$) durante todo o processo comprova matematicamente a **ausência de sobreajuste (*overfitting*)**, demonstrando que o

modelo generalizou os conceitos sem memorizar cegamente as amostras de treinamento.

4.3 Evolução do Agendamento da Taxa de Aprendizado

Para assegurar estabilidade numérica no espaço de parâmetros de baixa dimensionalidade do LoRA ($r = 16, \alpha = 16$), utilizou-se um agendador com aquecimento linear (*Linear Warmup*) seguido por decaimento cosseno (*Cosine Decay Schedule*). A Figura 2 ilustra a evolução da taxa de aprendizado ao longo dos 426 passos.

Figura 2 – Agendamento da Taxa de Aprendizado (*Cosine Decay with Warmup*).



Fonte: Elaborado pelo autor (2026).

O protocolo de agendamento comportou-se da seguinte forma:

- **Aquecimento (Passos 0 a 25):** A taxa de aprendizado é incrementada linearmente de zero até o valor máximo $\eta_{max} = 2,0 \times 10^{-4}$ (0,0002). Essa estratégia reduz o impacto de possíveis oscilações nos gradientes durante os primeiros lotes, proporcionando uma adaptação mais gradual dos parâmetros dos adaptadores.
- **Dcaimento Cosseno (Passos 25 a 426):** A partir do pico, a taxa de aprendizado é reduzida continuamente seguindo uma curva cosseno, atingindo $1,0 \times 10^{-4}$ no passo 225 ($\approx 1,0$ época) e chegando a $1,21 \times 10^{-8}$ no passo 425. Essa redução gradual permite que o otimizador AdamW realize atualizações cada vez menores ao longo do treinamento, contribuindo para a estabilização da função de perda e para o ajuste final dos parâmetros do modelo.

4.4 Avaliação Qualitativa dos Pareceres e Supressão de Alucinações via RAG

[ANOTAÇÃO: Aqui a intenção é desmonstrar a importância do RAG para evitar alucinações e mostrar a diferença resultante que a definição de regras de negócio tem após implementação, além da flexibilidade de atualizar o modelo com novas regras sem precisar re-treinar.]

Tabela 2 – Exemplo de Entrada de Dados e Parecer Estruturado Gerado pelo Modelo

Dados de Entrada do Proponente (Payload de Requisição):
Valor Solicitado: US\$ 20.000,00 Prazo: 36 meses Renda Anual: US\$ 58.000,00 Finalidade: Consolidacao de dividas Moradia: Imovel financiado Emprego: 10+ anos DTI: 22,51% FICO Score: 705 Atrasos (2 anos): 1 Consultas (6 meses): 0 Utilizacao de Rotativo: 49,40% Contas Abertas: 10 Registros Publicos: 1
Parecer Técnico Gerado pelo CreditMaster (Saída Estruturada em JSON):
<pre>{ "recomendacao": "Negar", "risco": "elevado", "parecer": "Recomendacao: Negar. Risco estimado elevado. O proponente apresenta score adequado (705), DTI de 22,51%, renda anual de US\$ 58.000,00 e solicitacao de US\$ 20.000,00 para consolidacao de dividas. Pontos favoraveis: nao ha pontos fortes relevantes nos campos analisados. Pontos de atencao: historico recente de inadimplencia e presenca de registro publico. O contrato historico evoluiu para perda, portanto o perfil deve ser recusado ou submetido a mitigadores fortes de garantia." }</pre>

Fonte: Elaborado pelo autor (2026).

5 CONCLUSÃO

[ANOTAÇÃO: Escrita será realizada após a implementação do RAG e validação do modelo.]

REFERÊNCIAS

ARACI, D. Finbert: Financial sentiment analysis with pre-trained language representations. **arXiv preprint arXiv:1908.10063**, 2019.

KHANDANI, A. E.; KIM, A. J.; LO, A. W. Consumer credit-risk models via machine-learning algorithms. **Journal of banking & finance**, Elsevier, v. 34, n. 11, p. 2767–2787, 2010.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, Nature Publishing Group, v. 521, n. 7553, p. 436–444, 2015.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. In: **Advances in neural information processing systems**. [S.l.: s.n.], 2017. p. 5998–6008.